Reinforcement Learning for Quantum Control

Morten Hjorth-Jensen

March 17, 2026

Outline

1. Why reinforcement learning?
2. Basic ingredients: states, actions, rewards, policies
3. Value functions and Bellman equations
4. Q-learning, policy gradients, actor–critic
5. Worked RL example with equations and pseudocode
6. Model-aware reinforcement learning
7. Reinforcement learning for quantum control
8. Applications to quantum sensing if time

Motivation

Many problems in physics can be phrased as optimization problems:

- variational calculations,
- control of dynamical systems,
- adaptive measurements,
- maximizing sensitivity in quantum sensors.

However, in many realistic settings:

- the objective is not known analytically,
- gradients are unavailable or noisy,
- decisions must be taken sequentially,
- present choices affect future opportunities.

Reinforcement learning (RL) addresses precisely this type of problem.

References

- R. Sutton and A. Barto, *Reinforcement Learning: An Introduction*, `https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf`
- Marin Bukov and Florian Marquardt, *Reinforcement Learning for Quantum Technology*, arXiv:2601.18953.
- Federico Belliardo, Fabio Zoratti, Florian Marquardt, and Vittorio Giovannetti, *Model-aware reinforcement learning for high-performance Bayesian experimental design in quantum metrology*, Quantum **8**, 1555 (2024).

My more narrow motivation (another talk perhaps)

- Quantum sensors permit fine-grained control over probe evolution and measurement settings.
- The central design problem is sequential: which control should be applied *next*, given what has already been learned?
- For realistic platforms, the estimation stack contains many non-smooth components: sampling of measurement outcomes, particle-filter resampling, and optional weak-measurement backaction.
- The article by Belliardo *et al.* proposes a model-aware RL framework that treats experimental design as a trainable control problem while retaining a faithful physical model.

Main thesis of the paper of Belliardo et al.

Core idea

Use a differentiable simulation of the experiment, Bayesian inference by particle filtering, and policy optimization to learn adaptive or non-adaptive control laws for quantum metrology.

- **Bayesian mode:** optimize the final estimation error after a complete measurement sequence.
- **Frequentist mode:** optimize the Cramer-Rao bound through the Fisher information matrix.
- **Model-aware:** gradients are propagated through a known sensor model rather than inferred only from trial-and-error interaction.

The measurement loop as a control architecture

1. From the current posterior summary, the agent outputs the next control x_{t+1}.
2. The probe evolves and is measured, producing a stochastic outcome y_{t+1}.
3. The Bayesian posterior is updated and fed back to the agent.

Adaptive policies are written as

$$x_{t+1} = F_\lambda\big(P(\theta \mid x_t, y_t);\ y_t;\ R_t;\ t\big),$$

while non-adaptive strategies suppress dependence on the posterior and outcomes:

$$x_{t+1} = F_\lambda(R_t, t).$$

- λ: trainable parameters of the policy (typically a neural network).
- R_t: resources consumed up to step t (time, photons, signal amplitude, ...).

Back to basic RL: What is Reinforcement Learning?

An agent interacts with an environment over discrete time steps:

$$s_0, a_0, r_1, s_1, a_1, r_2, \ldots$$

Core ingredients:

- **State** s_t: information available at time t,
- **Action** a_t: decision taken by the agent,
- **Reward** r_{t+1}: feedback from the environment,
- **Policy** $\pi(a|s)$: rule for choosing actions.

Goal:

$$\max_{\pi} \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t r_{t+1}\right]$$

where $0 \leq \gamma < 1$ is a discount factor.

Sequential Decision Making

In ordinary optimization we choose parameters once.
In reinforcement learning we choose a *sequence* of actions:

$$a_0, a_1, a_2, \ldots$$

This matters because:

- actions change the future state,
- the future state changes which actions are useful later,
- rewards may be delayed.

Thus RL is best viewed as **optimization over trajectories or strategies**.

Markov Decision Process

The standard framework is a Markov Decision Process (MDP).
Elements:

- state space $\mathcal{S}$,
- action space $\mathcal{A}$,
- transition probability $P(s'|s,a)$,
- reward function $R(s,a)$ or $R(s,a,s')$.

Markov property:

$$P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1}|s_t, a_t)$$

This is analogous to stochastic evolution equations in statistical physics.

Return and Value Functions

Define the return from time t:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

The **state-value function** under policy π is

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$$

The **action-value function** is

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s,\ a_t = a]$$

The **advantage** is

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

Interpretation: how much better is action a than the typical action at state s?

Bellman Equation

The value function satisfies a recursive relation:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) \left[r(s,a,s') + \gamma V^\pi(s') \right]$$

This is the Bellman equation.
Interpretation:

$$\text{value} = \text{immediate reward} + \text{discounted future value}$$

This is a self-consistency relation, reminiscent of mean-field equations and iterative schemes in physics.

Optimal Bellman Equation

Define the optimal value

$$V^*(s) = \max_\pi V^\pi(s)$$

and optimal action-value function

$$Q^*(s,a) = \max_\pi Q^\pi(s,a).$$

Then

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a)\left[r(s,a,s') + \gamma V^*(s')\right]$$

and

$$Q^*(s,a) = \sum_{s'} P(s'|s,a)\left[r(s,a,s') + \gamma \max_{a'} Q^*(s',a')\right].$$

This is the basis for dynamic programming and Q-learning.

Value Iteration and Policy Iteration

Value iteration:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s,a)\left[r + \gamma V_k(s')\right]$$

Policy iteration:

1. Evaluate V^π for current policy,
2. Improve the policy using

$$\pi_{\text{new}}(s) = \arg\max_a Q^\pi(s,a)$$

These are exact if the model is known and the state-action spaces are manageable.

Q-Learning

When the model is unknown, we can learn directly from sampled transitions.
Q-learning update:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \Big[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \Big]$$

Here:

- α is the learning rate,
- the term in brackets is the temporal-difference error.

This is a stochastic approximation to the Bellman optimality equation.

Exploration vs Exploitation

A central issue in RL is the exploration–exploitation tradeoff.

- **Exploitation**: choose the best-known action,
- **Exploration**: try other actions to discover better strategies.

Common strategies:

- ε-greedy,
- softmax/Boltzmann exploration,
- entropy regularization.

This is analogous to balancing local minimization and global search in complex optimization landscapes.

Why Policy Gradient Methods?

Q-learning is natural for discrete action spaces.
But in physics and control problems we often have:

- continuous control amplitudes,
- stochastic policies,
- large state spaces,
- differentiable parameterizations.

Then it is natural to parameterize the policy directly:

$$\pi_\theta(a|s)$$

and optimize the expected return

$$J(\theta).$$

Policy Gradient Setup

Define

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$$

where $\tau = (s_0, a_0, s_1, a_1, \dots)$ is a trajectory and

$$R(\tau) = \sum_{t=0}^{\infty} \gamma^t r_{t+1}.$$

Trajectory probability:

$$P_\theta(\tau) = \rho(s_0) \prod_{t=0}^{\infty} \pi_\theta(a_t|s_t) P(s_{t+1}|s_t, a_t)$$

We want to compute

$$\nabla_\theta J(\theta).$$

Policy Gradient Theorem: Step 1

Start with

$$J(\theta) = \sum_{\tau} P_\theta(\tau) R(\tau).$$

Differentiate:

$$\nabla_\theta J(\theta) = \sum_{\tau} \nabla_\theta P_\theta(\tau)\, R(\tau).$$

Use the log-derivative identity:

$$\nabla_\theta P_\theta(\tau) = P_\theta(\tau) \nabla_\theta \log P_\theta(\tau).$$

Hence

$$\nabla_\theta J(\theta) = \sum_{\tau} P_\theta(\tau) \nabla_\theta \log P_\theta(\tau)\, R(\tau).$$

Therefore

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\nabla_\theta \log P_\theta(\tau)\, R(\tau) \right].$$

Policy Gradient Theorem: Step 2

Now expand the log trajectory probability:

$$\log P_\theta(\tau) = \log \rho(s_0) + \sum_{t=0}^{\infty} \log \pi_\theta(a_t|s_t) + \sum_{t=0}^{\infty} \log P(s_{t+1}|s_t, a_t).$$

Only the policy depends on θ, so

$$\nabla_\theta \log P_\theta(\tau) = \sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t|s_t).$$

Substituting,

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\left(\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t|s_t) \right) R(\tau) \right].$$

Policy Gradient Theorem: Step 3

We can refine this by assigning to each time step only the future return from that point onward:

$$G_t = \sum_{k=t}^{\infty} \gamma^{k-t} r_{k+1}.$$

Then

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t|s_t)\, G_t\right].$$

This is the basis of the REINFORCE algorithm.
A more compact form is

$$\nabla_\theta J(\theta) = \mathbb{E}_{s\sim d^\pi,\, a\sim\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a|s)\, Q^{\pi_\theta}(s,a)\right]$$

where $d^\pi(s)$ is the discounted state visitation distribution.

Variance Reduction and Baselines

The estimator can have high variance.
We may subtract a baseline $b(s)$ without changing the expectation:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a|s)\,(Q^\pi(s,a) - b(s))\right].$$

A natural choice is

$$b(s) = V^\pi(s),$$

which gives

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a|s)\,A^\pi(s,a)\right].$$

This motivates actor–critic methods.

REINFORCE Algorithm

Basic Monte Carlo policy gradient algorithm

For each episode:

1. Generate a trajectory using π_θ,
2. Compute returns G_t,
3. Update parameters:

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)\, G_t$$

Advantages:

- simple,
- direct,
- works with stochastic policies.

Disadvantage:

- large variance.

Actor–Critic Methods

Actor–critic combines:

- **Actor**: policy $\pi_\theta(a|s)$,
- **Critic**: estimate of value function $V_\phi(s)$.

Temporal-difference error:

$$\delta_t = r_{t+1} + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

Policy update:

$$\theta \leftarrow \theta + \alpha_\theta \nabla_\theta \log \pi_\theta(a_t|s_t)\, \delta_t$$

Critic update:

$$\phi \leftarrow \phi - \alpha_\phi \nabla_\phi \delta_t^2$$

This is often much more efficient than pure Monte Carlo policy gradients.

Worked RL Example: Grid World

Consider a particle moving on a 1D lattice:

$$x \in \{0, 1, 2, 3, 4\}.$$

Actions:

$$a \in \{\text{left}, \text{right}\}.$$

Goal:

- reach $x = 4$,
- avoid spending too many steps,
- possibly avoid a penalty region.

Reward example:

$$r = \begin{cases} +10, & x = 4, \\ -1, & \text{each intermediate step.} \end{cases}$$

This is simple enough to analyze explicitly, but already contains the core RL ideas.

Worked Example: Bellman Equations

If the policy is deterministic and always moves right, then

$$V(4) = 0$$

if we stop at the goal, and for $x < 4$,

$$V(x) = -1 + \gamma V(x+1).$$

Thus

$$\begin{aligned}
V(3) &= -1, \\
V(2) &= -1 + \gamma(-1), \\
V(1) &= -1 + \gamma(-1 - \gamma), \\
V(0) &= -1 + \gamma(-1 - \gamma - \gamma^2).
\end{aligned}$$

So even this toy model already illustrates how future consequences are encoded recursively.

Worked Example: Q-Learning Equations

Suppose the agent experiences

$$s_t = 1, \qquad a_t = \text{right}, \qquad r_{t+1} = -1, \qquad s_{t+1} = 2.$$

Then the update is

$$Q(1, \text{right}) \leftarrow Q(1, \text{right}) + \alpha \left[-1 + \gamma \max_a Q(2, a) - Q(1, \text{right}) \right].$$

If initially all $Q = 0$, then

$$Q(1, \text{right}) \leftarrow \alpha(-1).$$

Repeated experience gradually propagates information backward from the goal.

Worked Example: Pseudocode

```
  Initialize Q(s,a) arbitrarily
For each episode:
    Initialize state s
    While s is not terminal:
        Choose action a from Q using epsilon-greedy
        Take action a, observe reward r and next state s'
        Q(s,a) <- Q(s,a) + alpha *
                  [r + gamma * max_a' Q(s',a') - Q(s,a)]
        s <- s'
```

- This is model-free.
- No knowledge of transition probabilities is required.
- The environment itself generates the data.

Worked Example: Policy Gradient Pseudocode

```
Initialize policy parameters theta
For each episode:
Generate trajectory using pi_theta
For each time step t:
Compute return G_t
Update:
        theta <- theta + alpha *
                sum_t [grad_theta log pi_theta(a_t|s_t) * G_t]
```

With a baseline:

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \left(G_t - b(s_t)\right)$$

This becomes especially natural when actions are continuous.

Model-Free vs Model-Aware Reinforcement Learning

Two broad philosophies:

Model-free RL

- learn directly from transitions,
- no explicit model of $P(s'|s, a)$ is used,
- examples: Q-learning, REINFORCE.

Model-aware (or model-based) RL

- use or learn a model of the dynamics,
- plan using that model,
- combine prediction and control.

In physics, model-aware RL is often especially attractive because we frequently know at least part of the dynamics.

What is Model-Aware Reinforcement Learning?

In model-aware RL we assume access to:

- known transition probabilities $P(s'|s, a)$, or
- an approximate learned model $\hat{P}_\phi(s'|s, a)$.

Then we can:

- simulate hypothetical futures,
- estimate returns before acting,
- plan over multiple time steps,
- use differentiable dynamics when available.

This is close in spirit to standard computational physics:

$$\text{equations of motion} \Rightarrow \text{simulation} \Rightarrow \text{optimization.}$$

Benefits of Model-Aware RL

Advantages:

- better sample efficiency,
- ability to plan ahead,
- easier incorporation of prior physical knowledge,
- natural interface with control theory.

Disadvantages:

- model bias if the model is wrong,
- errors may accumulate during planning,
- learning an accurate model may be difficult.

Thus model-aware RL is especially useful when the dynamics are fairly well understood, but the control task remains hard.

Model-Aware RL and Physics

In many physics applications we know the dynamics:

$$\dot{x} = f(x, u)$$

or in quantum mechanics

$$i\hbar \frac{d}{dt} |\psi(t)\rangle = H(u(t)) |\psi(t)\rangle .$$

Then the environment is not a black box:

- we can simulate trajectories,
- estimate gradients,
- evaluate candidate policies offline.

Model-aware RL thus sits naturally between:

- optimal control,
- stochastic optimization,
- machine learning.

Transition to Quantum Systems

Quantum control problems are naturally sequential:

- apply pulse,
- observe state evolution,
- choose next pulse,
- optimize final performance.

This makes RL conceptually attractive.
At the same time, quantum dynamics are often known from the Schrödinger equation, which makes model-aware RL particularly relevant.

Quantum Control

Quantum dynamics:

$$i\hbar \frac{d}{dt} |\psi(t)\rangle = H(t) |\psi(t)\rangle$$

Controlled Hamiltonian:

$$H(t) = H_0 + \sum_k u_k(t) H_k$$

Typical goals:

- state preparation,
- gate synthesis,
- suppression of decoherence,
- enhancement of sensor performance.

RL Formulation for Quantum Control

Mapping:

- **State**: wavefunction, density matrix, or measurement record,
- **Action**: control amplitude or pulse choice,
- **Reward**: fidelity, energy, Fisher information, robustness.

Examples of terminal rewards:

$$R = |\langle \psi_{\text{target}} | \psi(T) \rangle|^2$$

or

$$R = F_Q(T)$$

for a quantum sensing task.

Quantum Sensors and Fisher Information

A quantum sensor estimates a parameter θ.
Precision is bounded by the quantum Cramér–Rao bound:

$$\Delta\theta \geq \frac{1}{\sqrt{\nu F_Q}}$$

where:

- F_Q is the quantum Fisher information,
- ν is the number of repetitions.

Thus maximizing F_Q is a natural control objective for reinforcement learning.

Worked Quantum Example: Two-Level Sensor

Consider a qubit sensing a magnetic field B:

$$H(t) = \frac{\omega_0}{2}\sigma_z + \frac{B}{2}\sigma_z + u_x(t)\sigma_x.$$

Interpretation:

- ω_0: bare splitting,
- B: unknown signal,
- $u_x(t)$: control field chosen by the agent.

Goal:

- accumulate phase information about B,
- keep coherence as long as possible,
- maximize final sensitivity.

Quantum Sensor as an RL Problem

Discretize time into steps $t_1, \ldots, t_N$.
Possible actions:

$$a_t \in \{-u, 0, +u\}.$$

Possible state representation:

- Bloch vector $(\langle\sigma_x\rangle, \langle\sigma_y\rangle, \langle\sigma_z\rangle)$,
- time step,
- previous measurement outcomes.

Possible reward:

$$R = F_Q(T)$$

or

$$R = \left| \frac{\partial}{\partial B} \langle\sigma_y(T)\rangle \right|.$$

Why RL for Quantum Sensors?

RL can learn how to:

- alternate between free precession and control,
- suppress unwanted evolution,
- balance phase accumulation against decoherence,
- adapt measurements based on partial information.

The learned strategy may resemble:

- Ramsey interferometry,
- spin echo,
- dynamical decoupling,
- adaptive phase estimation.

Model-Aware RL for Quantum Control

Quantum systems are a prime example of model-aware RL.
If the Hamiltonian is known, we can:

- simulate trajectories offline,
- evaluate hypothetical pulse sequences,
- use differentiable propagators,
- combine RL with standard optimal-control ideas.

This is especially attractive for:

- quantum control,
- calibration,
- sensor design,
- adaptive metrology.

Summary

Reinforcement learning provides:

- a framework for sequential decision making,
- value-based and policy-based algorithms,
- a natural extension of stochastic optimization,
- direct relevance for quantum control and quantum sensing.

For physicists, the key conceptual bridge is:

$$\text{variational optimization} + \text{stochastic dynamics} + \text{control} \;\Rightarrow\; \text{reinforcement learning.}$$